Mark	

Team name	*A10*
Homework number	*HOMEWORK 07*
Due date	06/11/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understanding the logic and implementing the code			

Project name	7A - ADC scan using DMA		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To acquire the 3 required voltages (potentiometer, temperature sensor, and Vref) and send them to a remote terminal, we first consulted the "Hands-on Lab Schematics.pdf", verifying that the potentiometer is physically connected to pin PA1. In STM32CubeMX, we configured PA1 in ADC mode and enabled the internal channels for the Temperature Sensor and Vrefint. The ADC was set to operate with DMA in Circular Mode with the Scan Conversion Mode and DMA Continuous Requests enabled, setting the Number of Conversions to 3. The conversions were set to be triggered by Timer 2. We then assigned Rank 1, Rank 2, and Rank 3 to the respective parameters, setting a sampling time of 480 cycles for each, and enabled the ADC global interrupt. To manage the timing, Timer 2 (TIM2) was set up to generate an Update Event. Finally, the USART peripheral was configured by enabling its global interrupt and setting up the DMA for transmission (TX).

Part 2: Code Generation and Verification

After generating the project, we verified the TIM2 initialization parameters in the main.c file. By setting the prescaler to 8400-1 and the period to 10000-1, the timer is configured to generate a trigger event exactly every 1 second. In the main() function, immediately before the infinite loop, the system is activated by starting the timer count with HAL_TIM_Base_Start(&htim2) and launching the ADC in DMA mode using the HAL_ADC_Start_DMA function, passing an array of size 3 to store the data.

Part 3: Functional Logic Implementation

The entire functional logic of the system is handled entirely in the background, leaving the while(1) loop completely empty. The timing is strictly dictated by Timer 2 (TIM2), which automatically generates a hardware trigger (Update Event) exactly every 1 second. This event directly initiates the ADC to perform a scan sequence of the three configured channels (Vrefint, Potentiometer, and Temperature Sensor).

As the ADC completes each individual conversion, the DMA controller operates independently to transfer the 12-bit raw digital values directly from the ADC peripheral into a 3-element integer array (adc_buffer) in the microcontroller's memory. Once the full sequence is successfully transferred, the system triggers the HAL_ADC_ConvCpltCallback interrupt routine.

Inside this callback, the code processes the raw data: the potentiometer and reference voltages are calculated by scaling the 12-bit raw values (0-4095) to the 3.3V reference using the formula (raw_value * 3.3 / 4095). For the internal temperature, we calculate the sensor's voltage (Vsense) and then apply the specific formula ((Vsense - 0.76) / 0.0025) + 25.

Finally, the values are formatted into a string using snprintf and transmitted to the remote terminal without blocking the CPU by calling HAL_UART_Transmit_DMA.

Professor comments:

Project name	7B - Light Dependent Resistor (LDR)		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the goal of acquiring the Light Dependent Resistor (LDR) value every 1 ms, calculating its average, and converting it to a lux level every 1 second, we first consulted the "Hands-on Lab Schematics.pdf" to identify the correct routing. We verified that the LDR is connected to the PA0 pin of the microcontroller. In STM32CubeMX, we configured PA0 as an analog input for the ADC. To enable background data collection, the ADC was set to operate with DMA in Circular Mode and DMA Continuous Requests enabled. The conversions were set to be triggered automatically by Timer 2. The USART2 peripheral was also configured in asynchronous mode with TX interrupt enabled to transmit the final calculated data to the remote terminal.

Part 2: Code Generation and Verification

After generating the project code, we verified the initialization parameters within the main.c file. For Timer 2, which dictates the sampling rate, the prescaler was set to 84-1 and the counter period (ARR) to 1000-1. Assuming an 84 MHz clock, this specific configuration ensures that the timer generates an "Update Event"

exactly every 1 millisecond, continuously triggering the ADC without CPU intervention. In the main() function, immediately before the empty infinite loop, the system is armed by starting Timer 2 with HAL_TIM_Base_Start(&htim2) and launching the ADC in DMA mode with HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, 2000). A buffer size of 2000 was chosen to hold exactly 2 seconds worth of data.

Part 3: Functional Logic Implementation

The entire functional logic leverages the DMA Ping-Pong buffering technique, utilizing both the Half-Complete and Complete callbacks to ensure continuous sampling without data loss or CPU stalling. Every millisecond, the hardware timer triggers the ADC, and the DMA fills the adc_buffer array sequentially in the background.

When the buffer reaches its halfway point (1000 samples, representing exactly 1 second of data), the HAL_ADC_ConvHalfCpltCallback interrupt is automatically triggered. Inside this routine, a custom function is called to process the first half of the array. It calculates the raw average of the 1000 samples, scales it to a voltage value using the 3.3V reference, and derives the LDR resistance (in Ohms) using the voltage divider formula. Finally, it applies the specific non-linear characteristic equation to convert the resistance into a luminosity value in Lux. This data is then formatted into a string using snprintf and transmitted to the terminal via the non-blocking HAL_UART_Transmit_DMA function.

Exactly one second later, when the buffer is completely full, the HAL_ADC_ConvCpltCallback fires, performing identical calculations and transmissions for the second half of the buffer. Because the DMA operates in Circular Mode, it immediately wraps around to overwrite the beginning of the buffer, allowing this cycle to repeat infinitely and seamlessly satisfying the requirement to acquire data every 1 ms and output the processed lux level every 1 second.

Professor comments: